Applied Machine Learning

Boosting

UNIVERSITY OF
GOTHENBURG

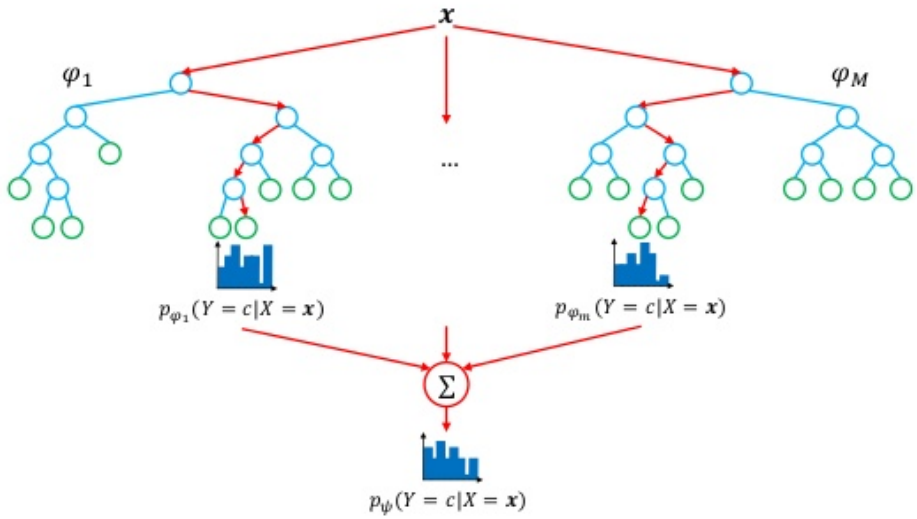
CHALMERS

Richard Johansson

richard.johansson@cse.gu.se

ensembles: recap

- we have seen that **ensembles** can be more robust than single models (particularly for trees)



- we saw some **ensemble building** methods such as bagging
- today: an approach that builds ensembles **iteratively**

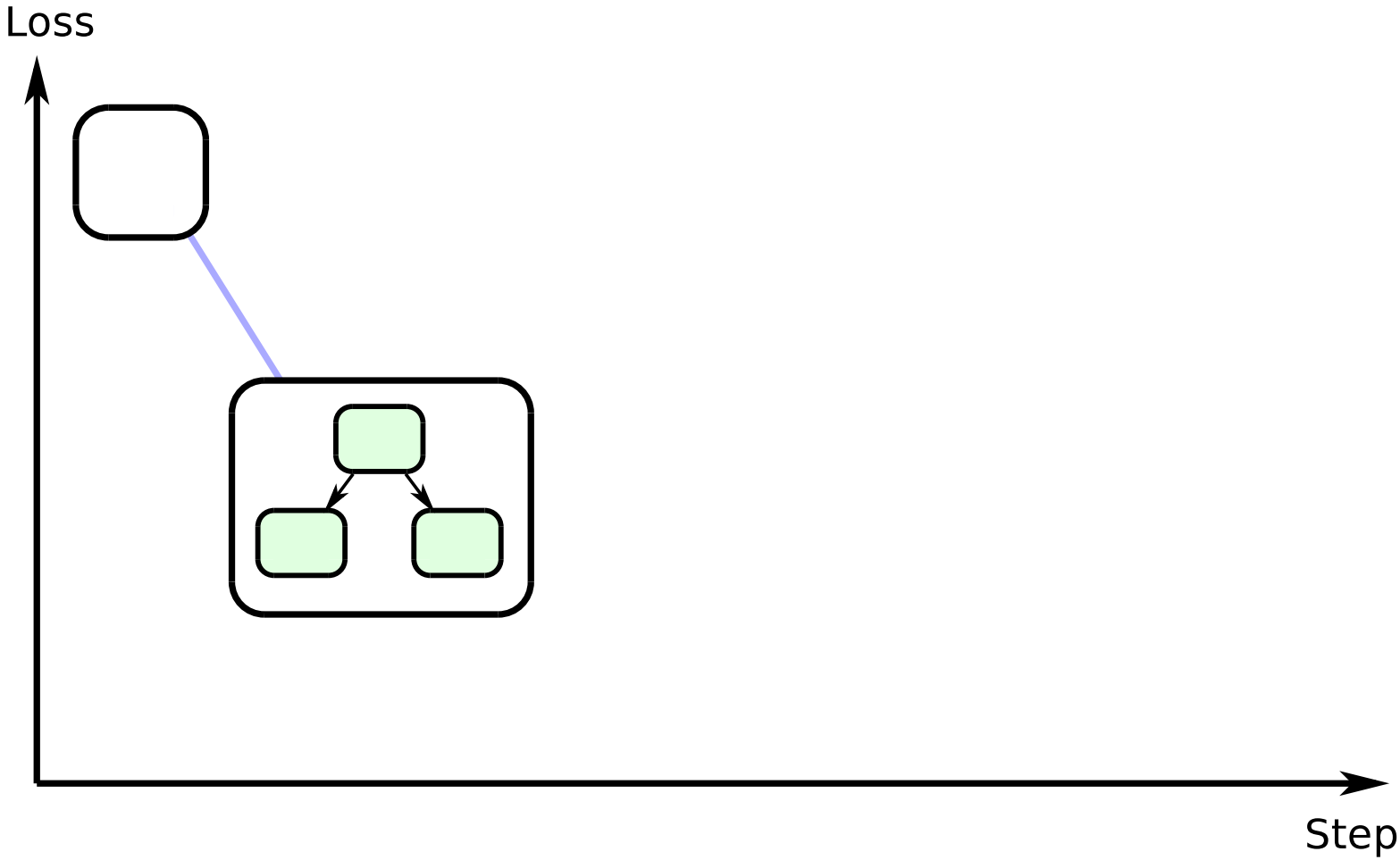
can we try to fix the errors?

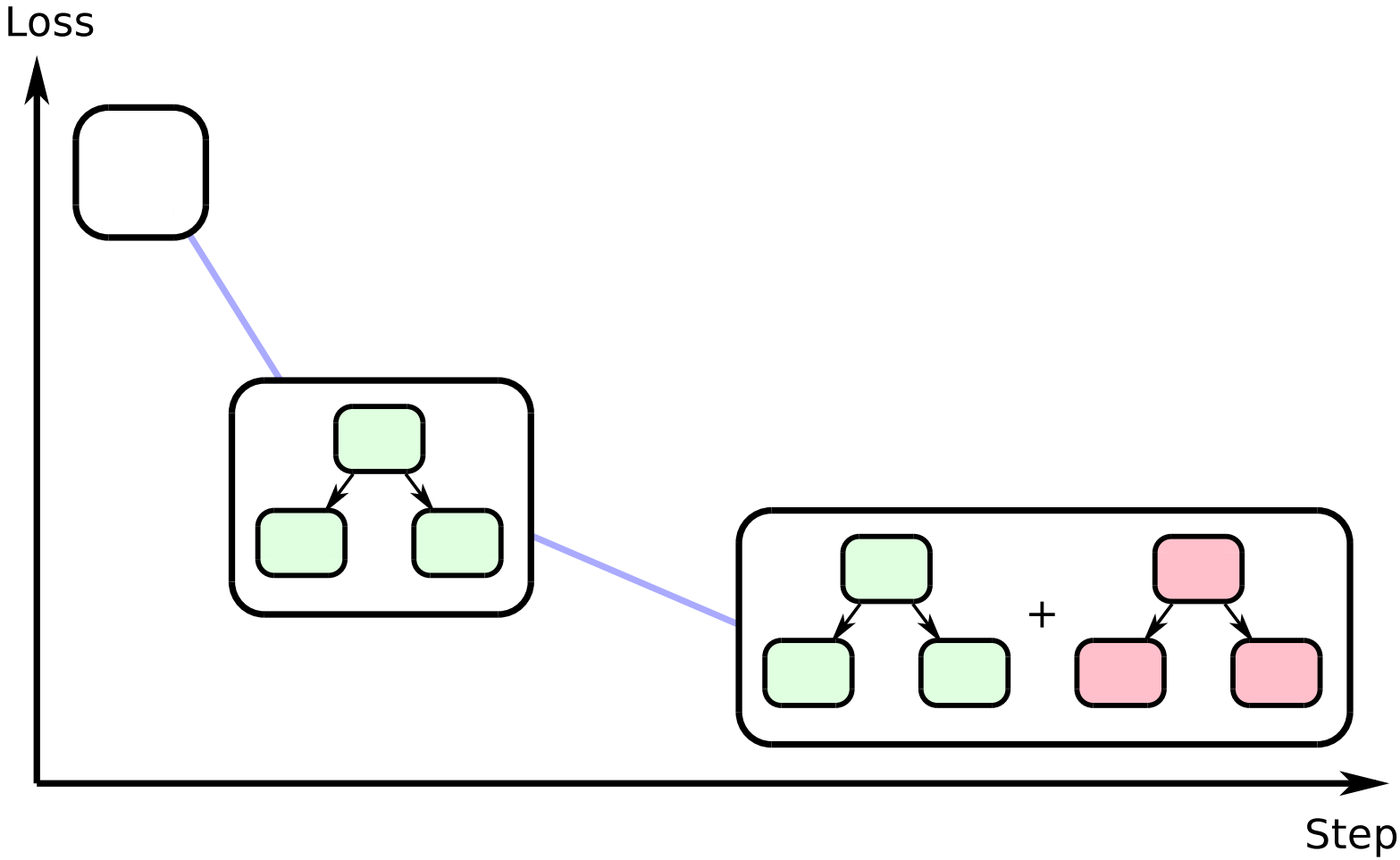
- assume that we have trained a classifier or regressor, and that it makes some mistakes on the training data
- can we try to train another model that "cleans up the mess"?

can we try to fix the errors?

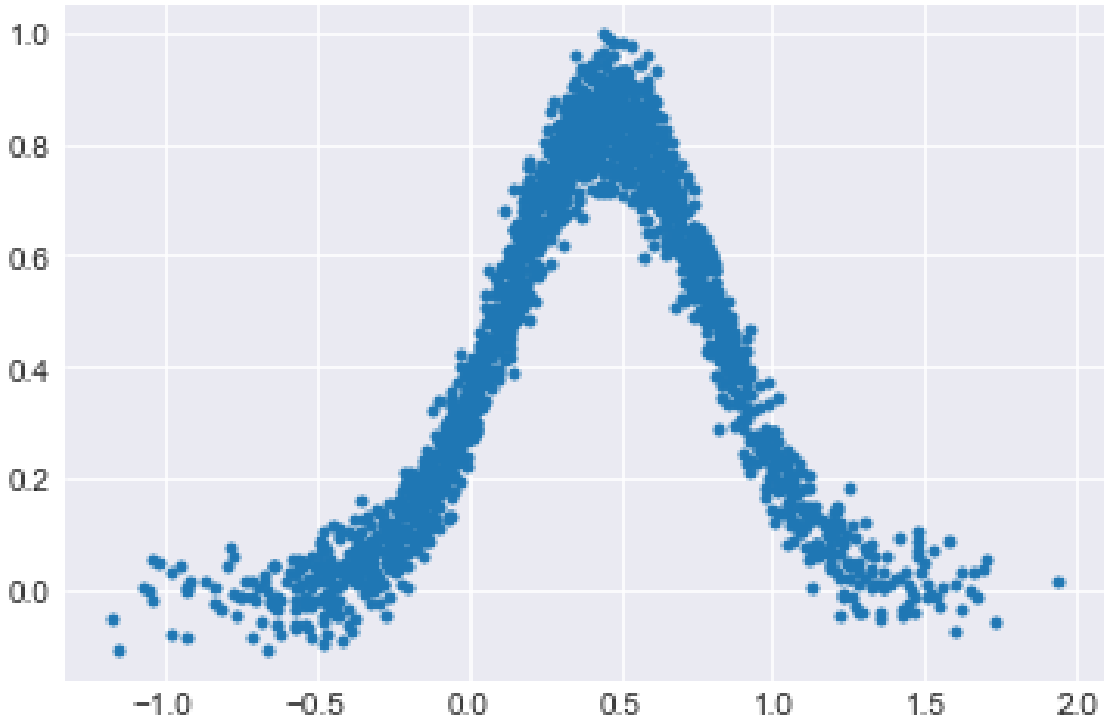
- assume that we have trained a classifier or regressor, and that it makes some mistakes on the training data
- can we try to train another model that "cleans up the mess"?
- yes, and this idea is called **boosting**
  - an ensemble where the sub-models are trained sequentially
  - to gradually improve the overall performance


Loss
Step

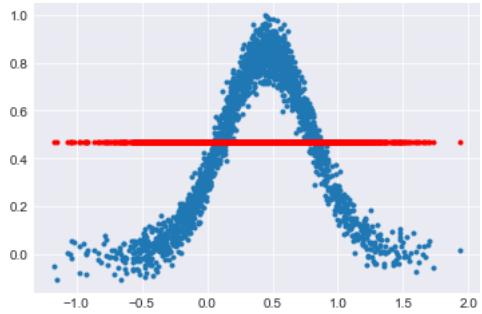

Loss
Step


Loss
+
Step

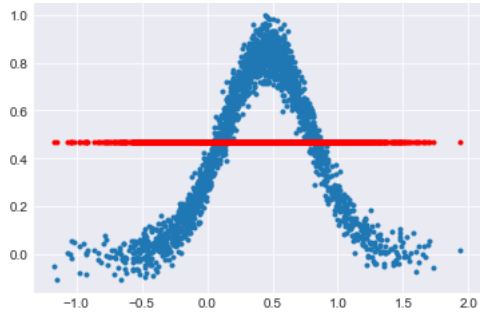
toy regression example

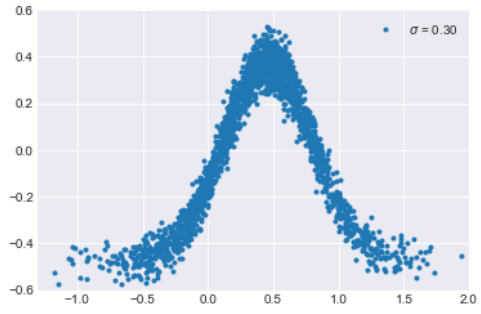


fixing the errors iteratively

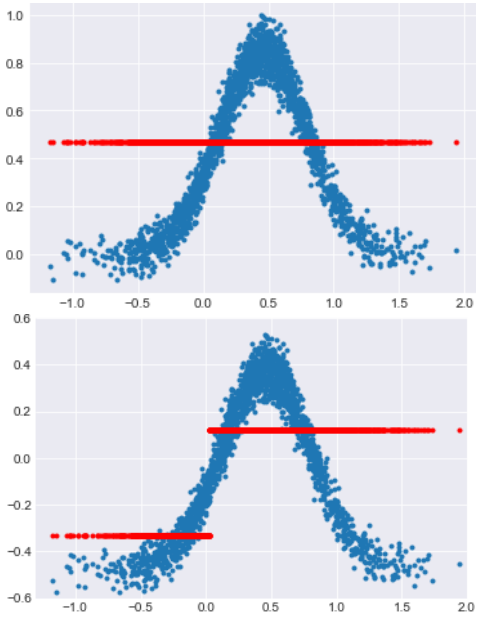


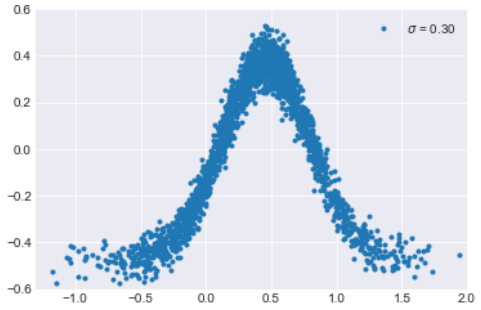
fixing the errors iteratively



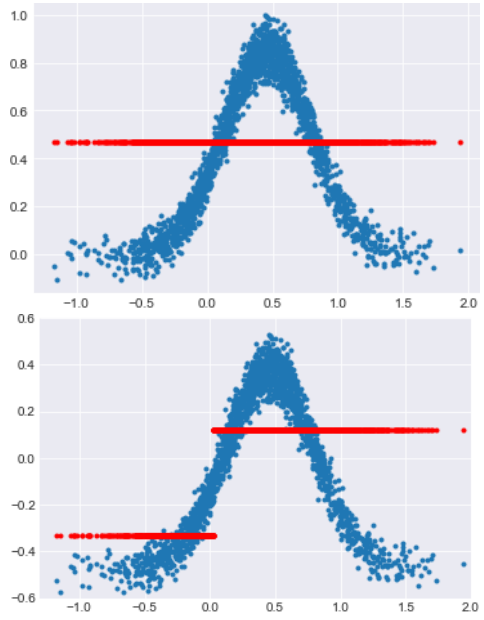


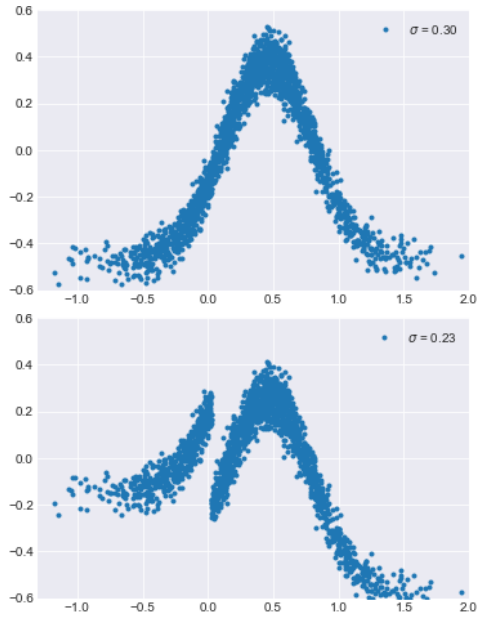
fixing the errors iteratively



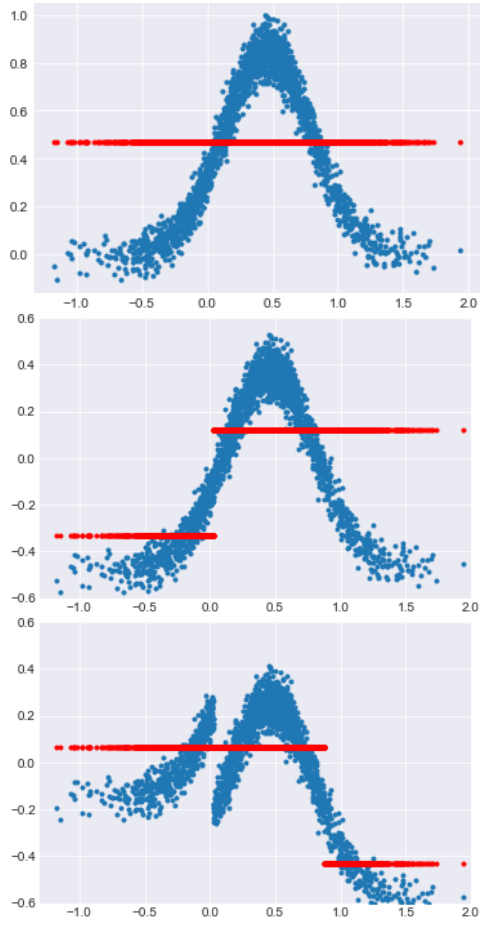


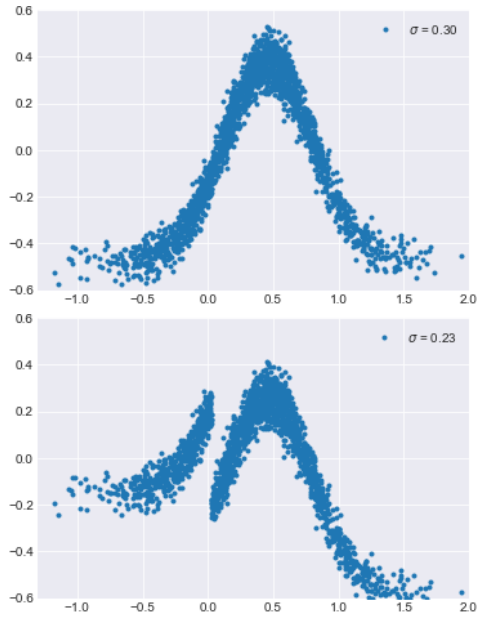
fixing the errors iteratively



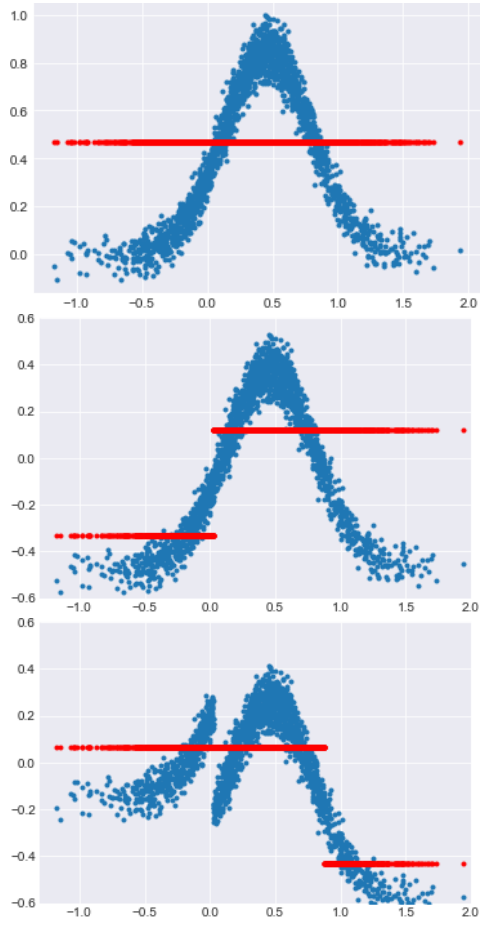


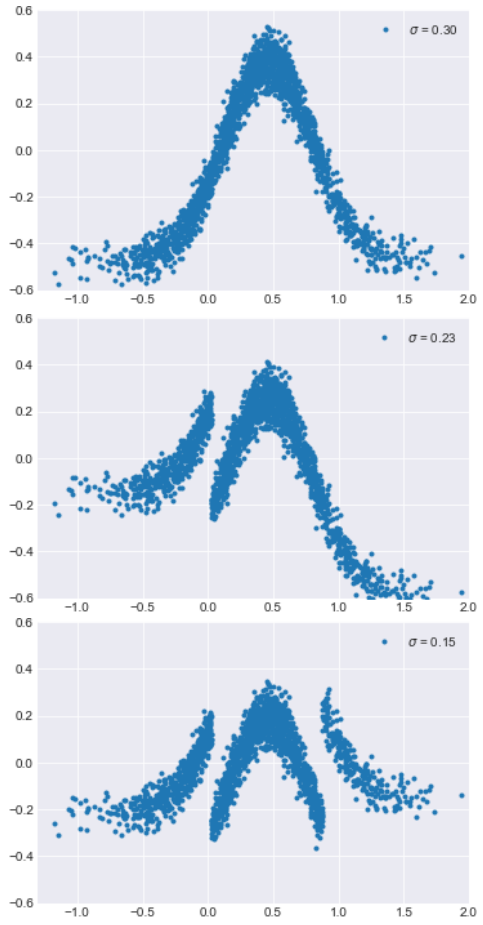
fixing the errors iteratively



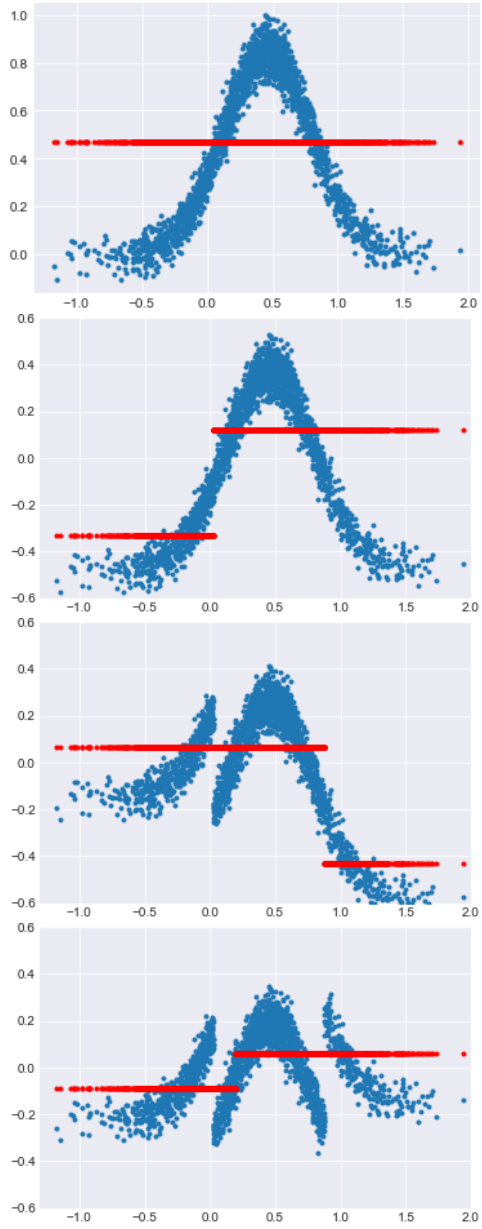


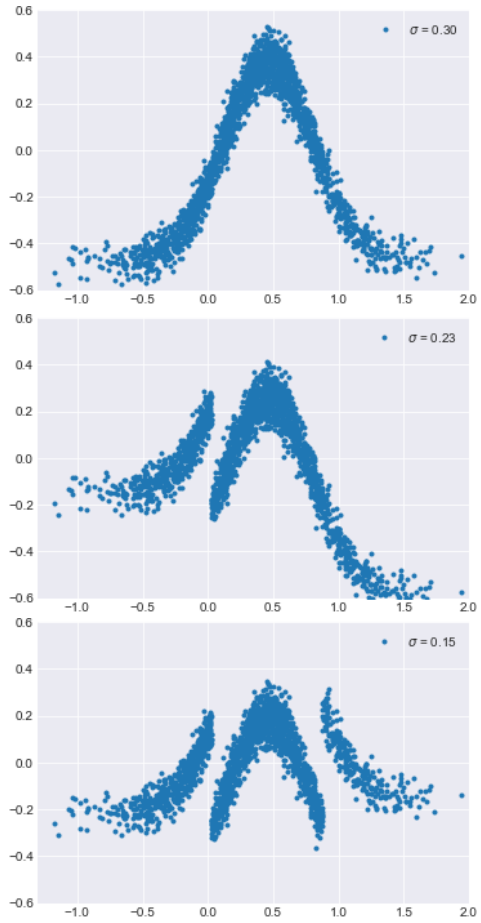
fixing the errors iteratively



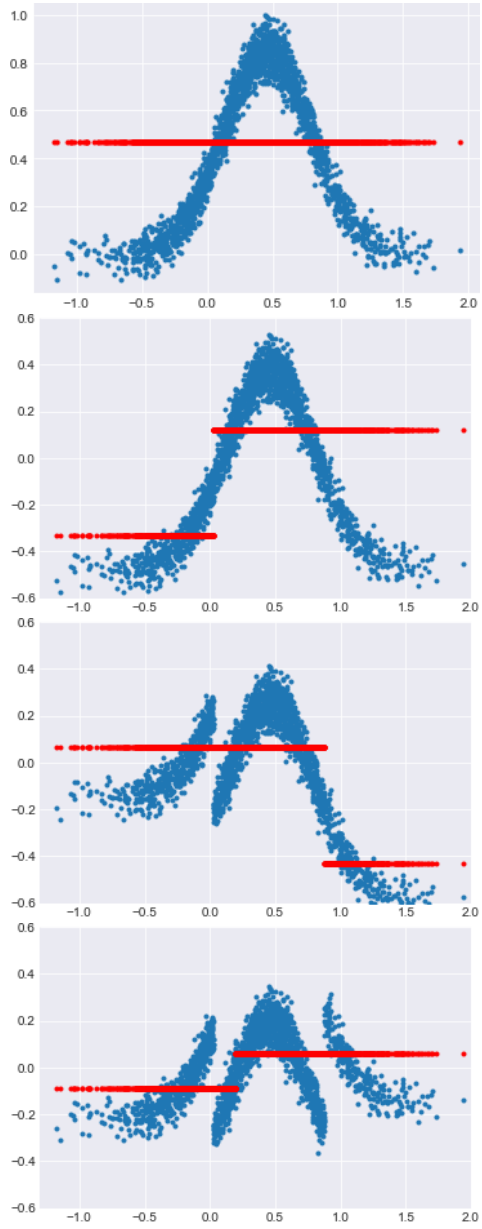


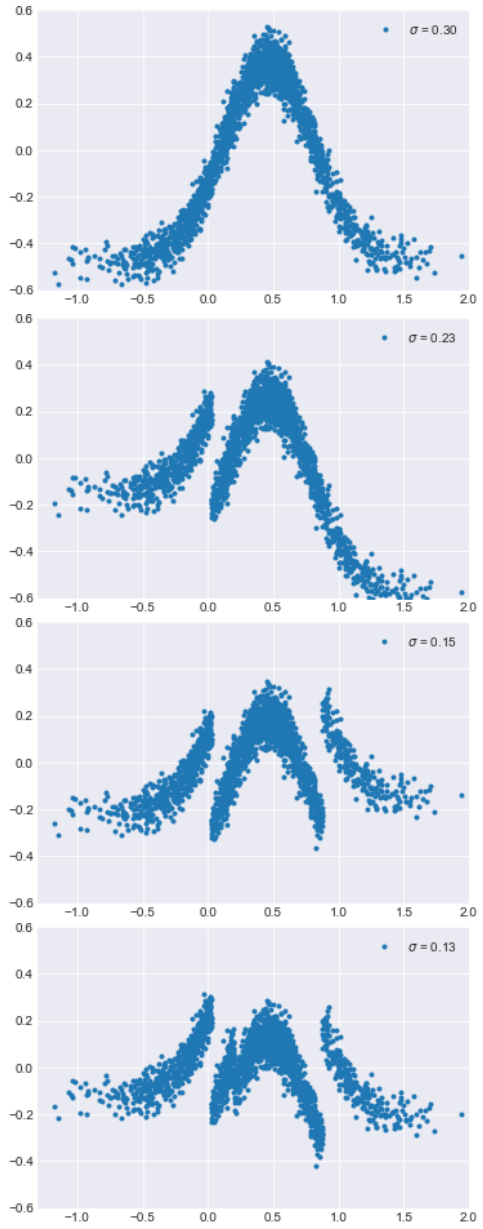
fixing the errors iteratively



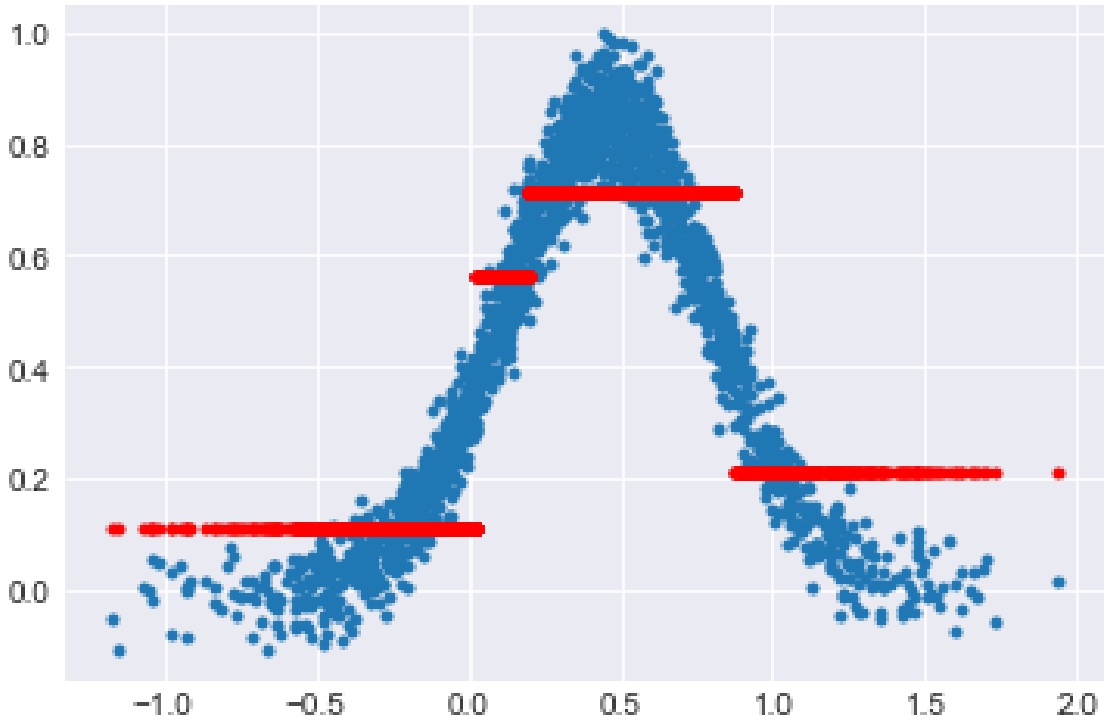


fixing the errors iteratively



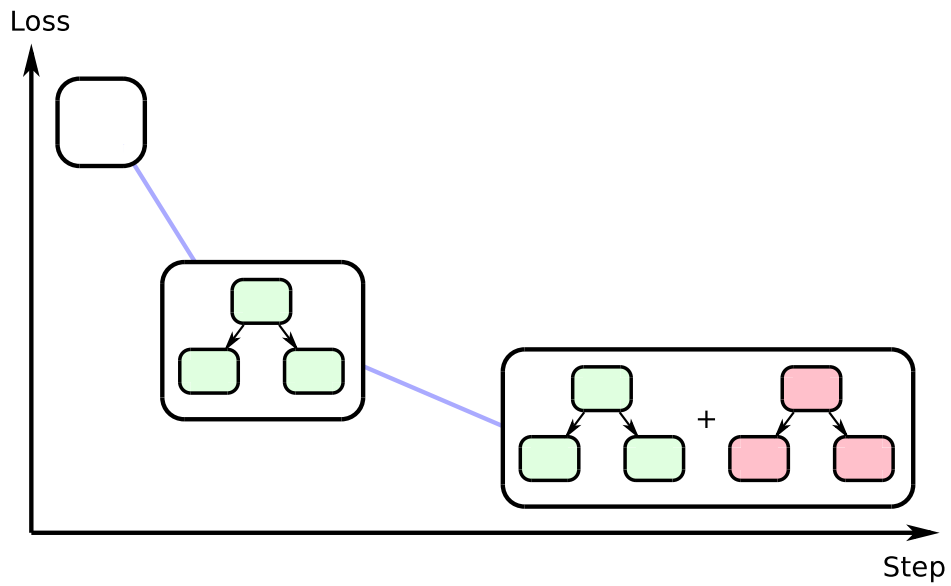


toy regression example: combining the pieces



approaches to boosting: overview

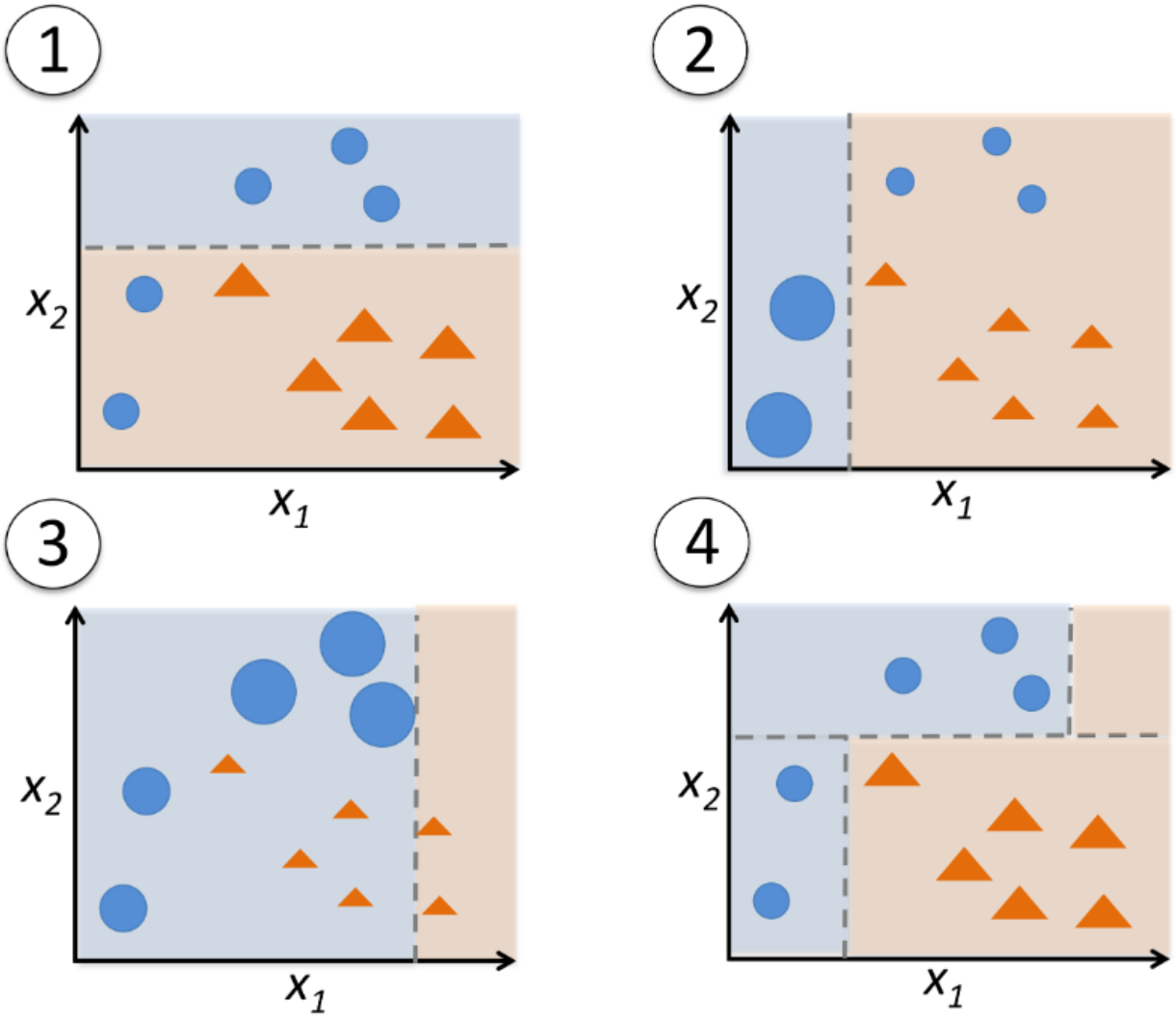
- **AdaBoost**: classical algorithm, works for classification tasks
- **gradient boosting**: generalizes AdaBoost, works for classification and regression



adaptive boosting: AdaBoost

- **AdaBoost** is the most famous boosting algorithm
- after each iteration, check which instances are misclassified
  - then give misclassified instances a higher importance before the next iteration
- this can be applied using any base classifier that can handle weighted instances
  - typical choice: small decision trees ("decision stumps")

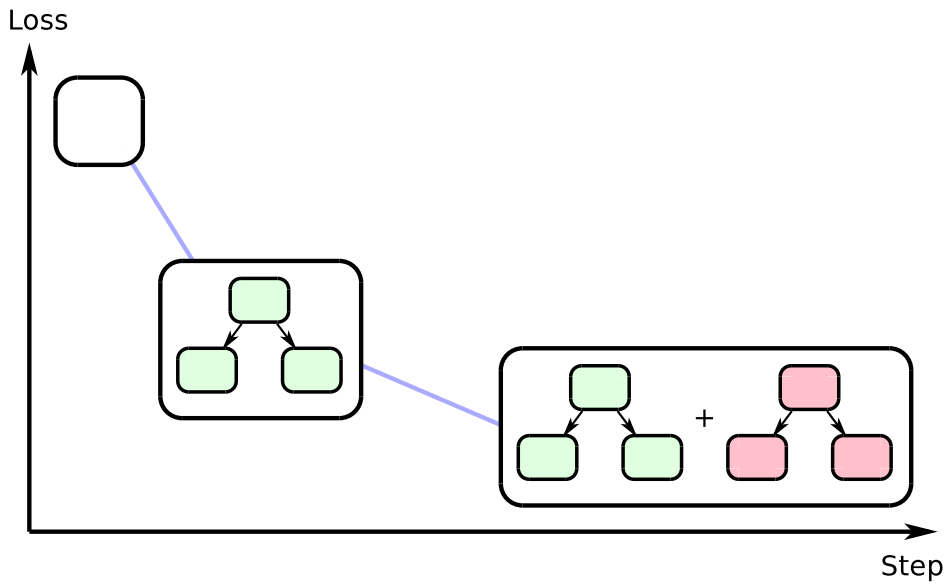
adaptive boosting: AdaBoost



[source]

gradient boosting

- **gradient boosting** generalizes AdaBoost
- idea: gradually add sub-models to minimize a loss function
- it is based on a principle of gradient descent



gradient boosting: pseudocode

let F_0 be a "dummy" constant model
for $m = 1, \ldots, M$
  for each pair $(\boldsymbol{x}_i, y_i)$ in the training set
    let r_i be the **pseudo-residual** $R(y_i, F_{m-1}(\boldsymbol{x}_i))$
  train a sub-model h_m on the pseudo-residuals
  create F_m by **adding** h_m to F_{m-1}
return F_M

gradient boosting: pseudocode

let F_0 be a "dummy" constant model
for $m = 1, \ldots, M$
  for each pair $(\boldsymbol{x}_i, y_i)$ in the training set
    let r_i be the **pseudo-residual** $R(y_i, F_{m-1}(\boldsymbol{x}_i))$
  train a sub-model h_m on the pseudo-residuals
  create F_m by **adding** h_m to F_{m-1}
return F_M

- in practice, the sub-models are typically trees

gradient boosting: pseudocode

let F_0 be a "dummy" constant model
for $m = 1, \ldots, M$
  for each pair $(\boldsymbol{x}_i, y_i)$ in the training set
    let r_i be the **pseudo-residual** $R(y_i, F_{m-1}(\boldsymbol{x}_i))$
  train a sub-model h_m on the pseudo-residuals
  create F_m by **adding** h_m to F_{m-1}
return F_M

- in practice, the sub-models are typically trees
- the **pseudo-residual** $R(y_i, F_{m-1}(\boldsymbol{x}_i))$ is defined as the negative gradient of the loss function

$$-\nabla_{\hat{y}} \text{Loss}(y, \hat{y})$$

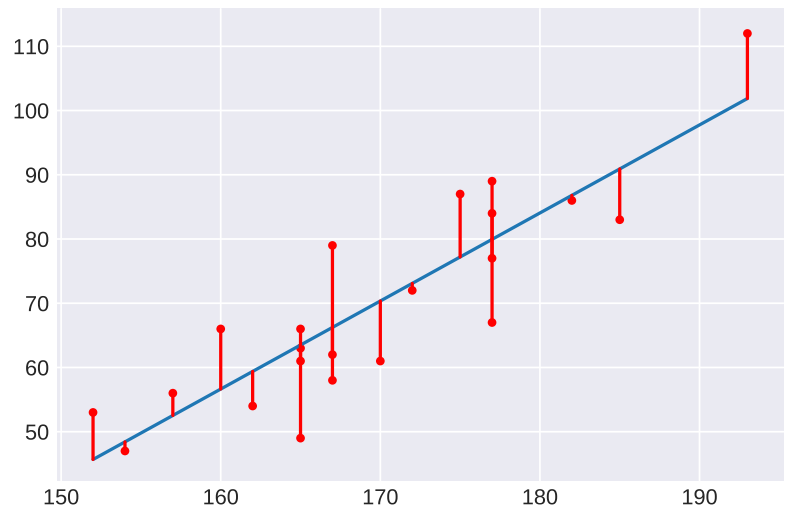
why "pseudo-residual"?

- for the squared error loss

$$\text{Loss}(y, \hat{y}) = (y - \hat{y})^2$$

the negative gradient is equal to (two times) the **residual**

$$-\nabla_{\hat{y}} \text{Loss}(y, \hat{y}) = 2(y - \hat{y})$$



adding a sub-model to the ensemble

1. first find the γ_m that minimizes

$$\sum_{i=1}^{n} \text{Loss}\left[y_i, F_{m-1}(\boldsymbol{x}_i) + \boldsymbol{\gamma_m} h_m(\boldsymbol{x}_i)\right]$$

adding a sub-model to the ensemble

1. first find the γ_m that minimizes

$$\sum_{i=1}^{n} \text{Loss}\left[y_i, F_{m-1}(\boldsymbol{x}_i) + \boldsymbol{\gamma_m} h_m(\boldsymbol{x}_i)\right]$$

2. γ_m is multiplied by a **learning rate** η to mitigate overfitting

main hyperparameters

sklearn.ensemble.GradientBoostingClassifier

class `sklearn.ensemble.` **`GradientBoostingClassifier`**(**, loss='deviance', learning_rate=0.1, n_estimators=100, subsample=1.0, criterion='friedman_mse', min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_depth=3, min_impurity_decrease=0.0, min_impurity_split=None, init=None, random_state=None, max_features=None, verbose=0, max_leaf_nodes=None, warm_start=False, validation_fraction=0.1, n_iter_no_change=None, tol=0.0001, ccp_alpha=0.0*) [source]

Gradient Boosting for classification.

GB builds an additive model in a forward stage-wise fashion; it allows for the optimization of arbitrary differentiable loss functions. In each stage `n_classes_` regression trees are fit on the negative gradient of the binomial or multinomial deviance loss function. Binary classification is a special case where only a single regression tree is induced.

Read more in the User Guide.

- **ensemble size** (`n_estimators`)
- **learning rate**
- as well as the usual decision tree hyperparameters. . .

gradient boosting implementations

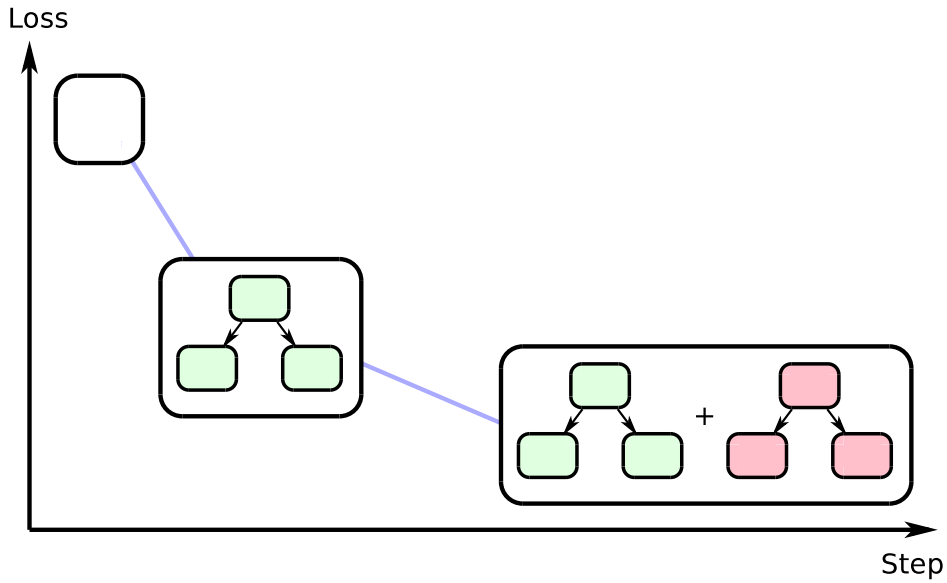
- scikit-learn (Python):
  - `sklearn.ensemble.GradientBoostingClassifier`
  - `sklearn.ensemble.GradientBoostingRegressor`
- **XGBoost** (multiple languages)
- **H2O** (multiple languages)
- ...

tree ensembles in Kaggle

- GB (and other tree-based ensembles) often dominate in competitions for various prediction tasks, such as Kaggle
- why?
  - robust to preprocessing
  - easy to mix different types of features, nice for tabular data
  - easy to tune
- especially the software XGBoost is popular in Kaggle

boosting: main points

- **boosting** algorithms build ensembles sequentially
- main idea: **correcting errors** from previous steps



- **gradient boosting** adds submodels based on a principle of gradient descent
- popular in industry and in competitions, mainly for tabular data